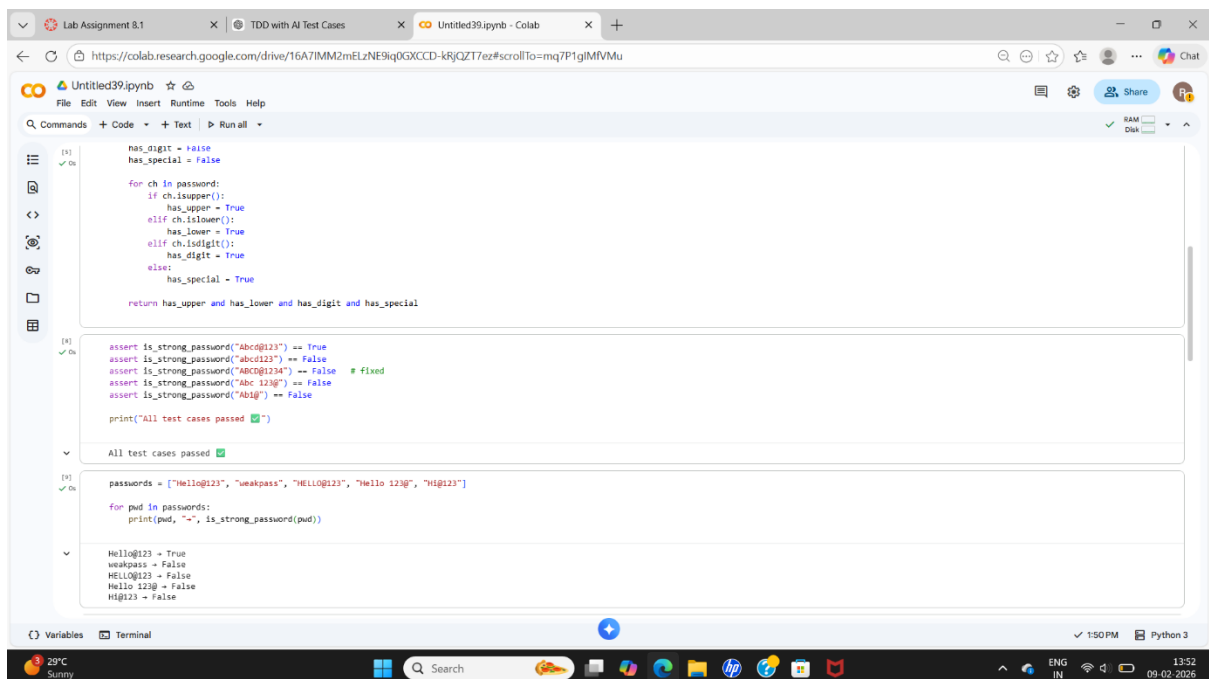




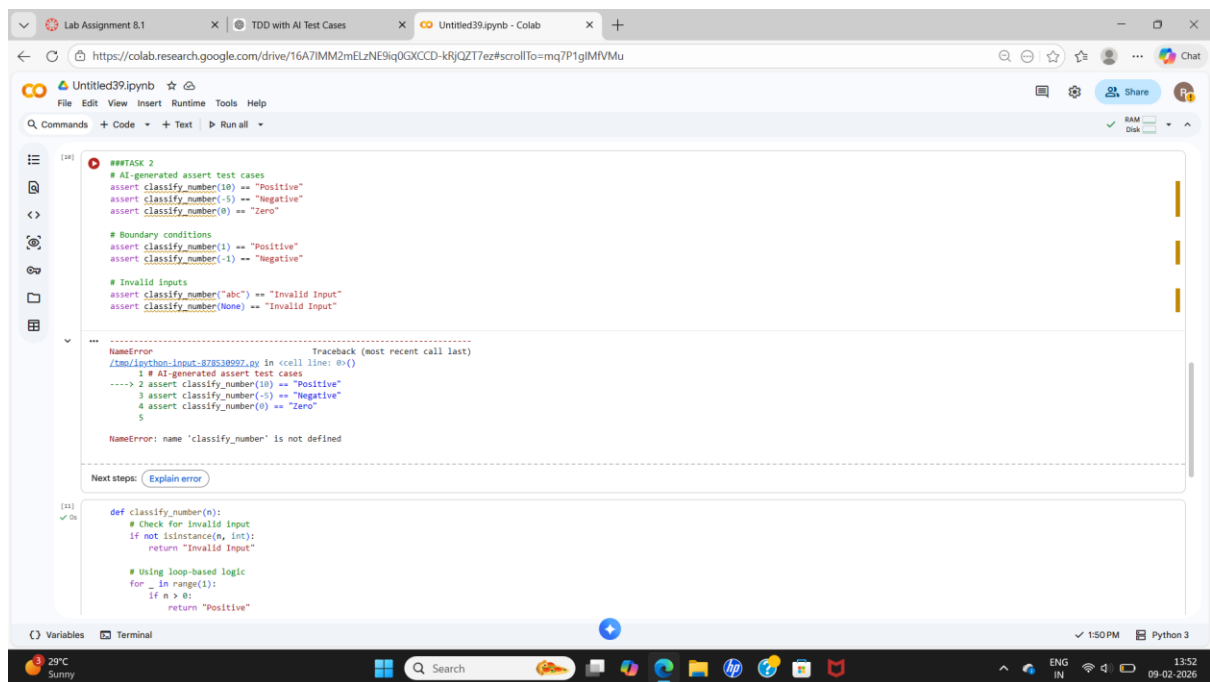
Code Explanation:

The function checks whether a password is strong by verifying length, presence of uppercase, lowercase, digit, special character, and absence of spaces. It returns True for strong passwords and False otherwise.

AI Explanation:

AI generated test cases for strong and weak passwords. These tests helped verify security rules and identify incorrect password patterns.

Task 2: Number Classification Using Loops



The screenshot shows a Google Colab notebook titled 'Untitled39.ipynb'. The code defines a function `classify_number(n)` that checks for invalid input and then uses a loop to classify the number. The function returns 'Invalid Input' for non-integers, 'Positive' for integers greater than 0, 'Negative' for integers less than 0, and 'Zero' for integers equal to 0. The notebook shows a `NameError` because the function is not defined before it is called in the test cases.

```
##TASK 2
# AI-generated assert test cases
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"

# Boundary conditions
assert classify_number(1) == "Positive"
assert classify_number(-1) == "Negative"

# Invalid inputs
assert classify_number("abc") == "Invalid Input"
assert classify_number(None) == "Invalid Input"

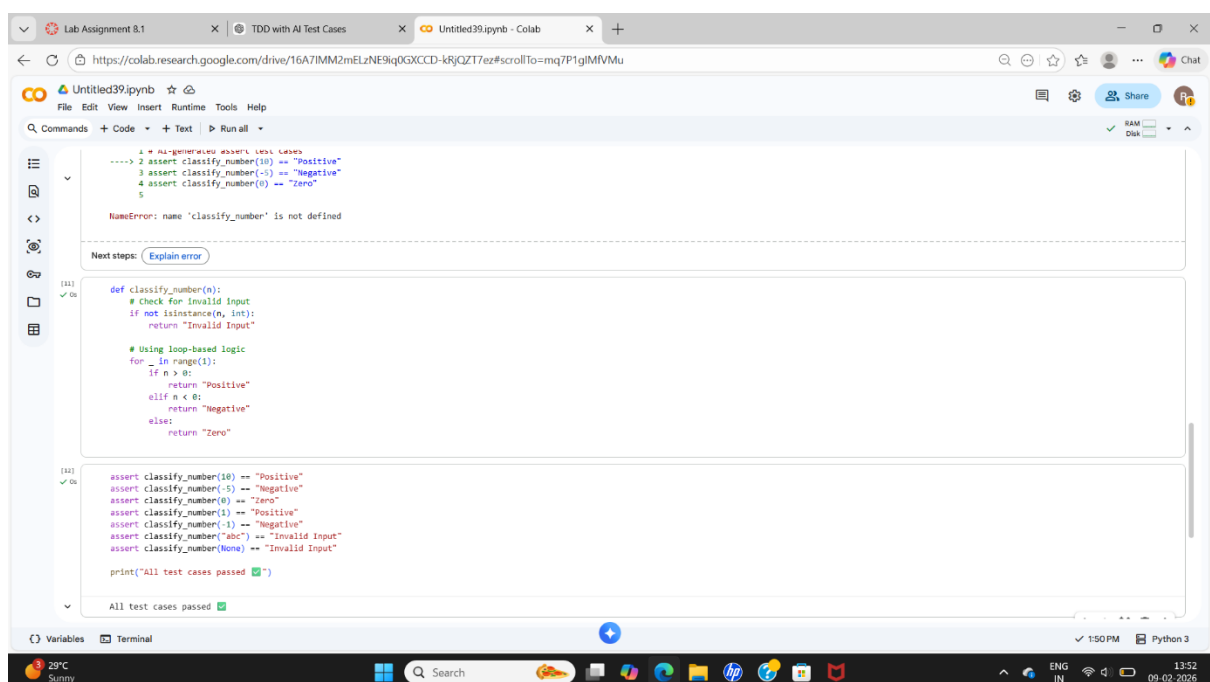
-----
Traceback (most recent call last)
/mnt/ipython-input-878538997.py in <cell line: 8>()
----> 1 # AI-generated assert test cases
      2 assert classify_number(10) == "Positive"
      3 assert classify_number(-5) == "Negative"
      4 assert classify_number(0) == "Zero"
      5

NameError: name 'classify_number' is not defined

Next steps: Explain error
```

```
[11] def classify_number(n):
    # Check for invalid input
    if not isinstance(n, int):
        return "Invalid Input"

    # Using loop-based logic
    for _ in range(1):
        if n > 0:
            return "Positive"
        elif n < 0:
            return "Negative"
        else:
            return "Zero"
```



The screenshot shows the same Google Colab notebook, but now the function `classify_number` is defined before the test cases. All test cases pass, and the notebook shows a green checkmark for each assertion.

```
1 # AI-generated assert test cases
----> 2 assert classify_number(10) == "Positive"
      3 assert classify_number(-5) == "Negative"
      4 assert classify_number(0) == "Zero"
      5

NameError: name 'classify_number' is not defined

Next steps: Explain error
```

```
[11] def classify_number(n):
    # Check for invalid input
    if not isinstance(n, int):
        return "Invalid Input"

    # Using loop-based logic
    for _ in range(1):
        if n > 0:
            return "Positive"
        elif n < 0:
            return "Negative"
        else:
            return "Zero"
```

```
[12] assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
assert classify_number(1) == "Positive"
assert classify_number(-1) == "Negative"
assert classify_number("abc") == "Invalid Input"
assert classify_number(None) == "Invalid Input"

print("All test cases passed ✅")

All test cases passed ✅
```

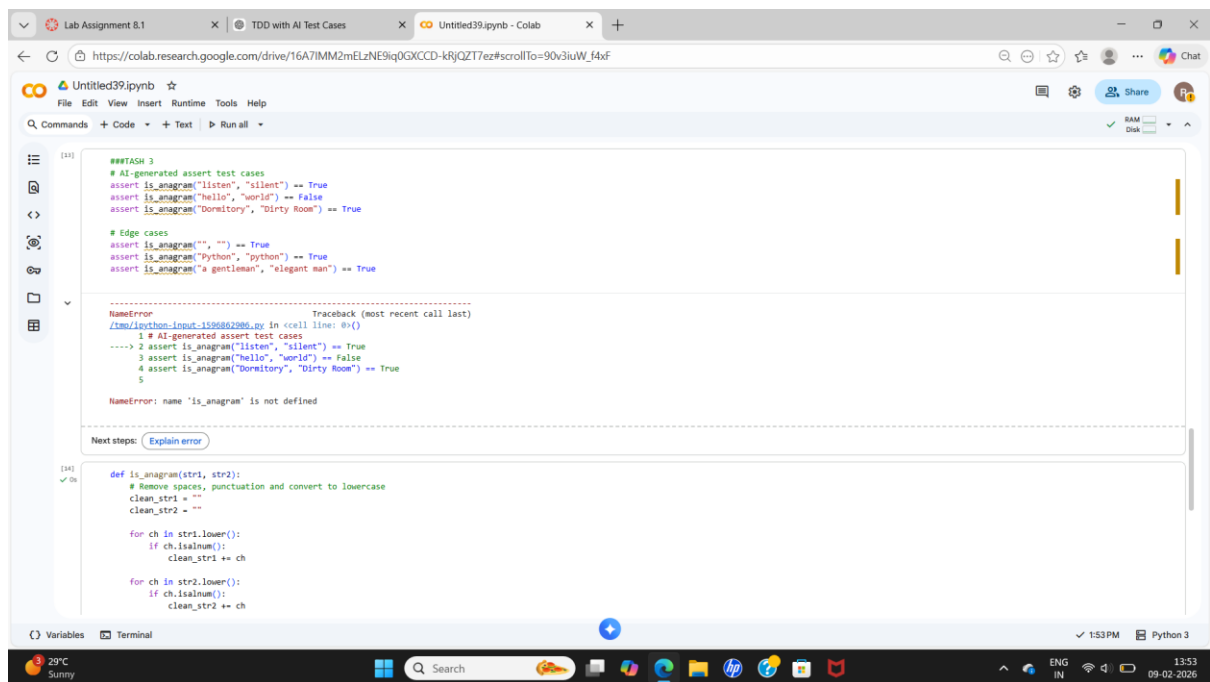
Code Explanation:

The function classifies a number as Positive, Negative, or Zero. It first checks for invalid inputs like strings or None and then uses logic inside a loop to return the correct classification.

AI Explanation:

AI generated test cases including boundary values (-1, 0, 1) and invalid inputs, ensuring correct classification.

Task 3: Anagram Checker



```
##TASK 3
# AI-generated assert test cases
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True

# Edge cases
assert is_anagram("", "") == True
assert is_anagram("python", "python") == True
assert is_anagram("a gentleman", "elegant man") == True

-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-1596862986.py in <cell line: 0>()
----> 2 assert is_anagram("listen", "silent") == True
      3 assert is_anagram("hello", "world") == False
      4 assert is_anagram("Dormitory", "Dirty Room") == True
      5

NameError: name 'is_anagram' is not defined

Next steps: Explain error

[14]:
def is_anagram(str1, str2):
    # Remove spaces, punctuation and convert to lowercase
    clean_str1 = ""
    clean_str2 = ""

    for ch in str1.lower():
        if ch.isalnum():
            clean_str1 += ch

    for ch in str2.lower():
        if ch.isalnum():
            clean_str2 += ch
```

```

NameError: name 'is_anagram' is not defined

Next steps: Explain error

[14]: def is_anagram(str1, str2):
      # Remove spaces, punctuation and convert to lowercase
      clean_str1 = ""
      clean_str2 = ""

      for ch in str1.lower():
          if ch.isalnum():
              clean_str1 += ch

      for ch in str2.lower():
          if ch.isalnum():
              clean_str2 += ch

      # If lengths differ, not anagrams
      if len(clean_str1) != len(clean_str2):
          return False

      # Compare sorted characters
      return sorted(clean_str1) == sorted(clean_str2)

[15]: assert is_anagram("listen", "silent") == True
      assert is_anagram("hello", "world") == False
      assert is_anagram("Dormitory", "Dirty Room") == True
      assert is_anagram("", "") == True
      assert is_anagram("Python", "python") == True
      assert is_anagram("a gentleman", "elegant man") == True

      print("All test cases passed ✅")

All test cases passed ✅

```

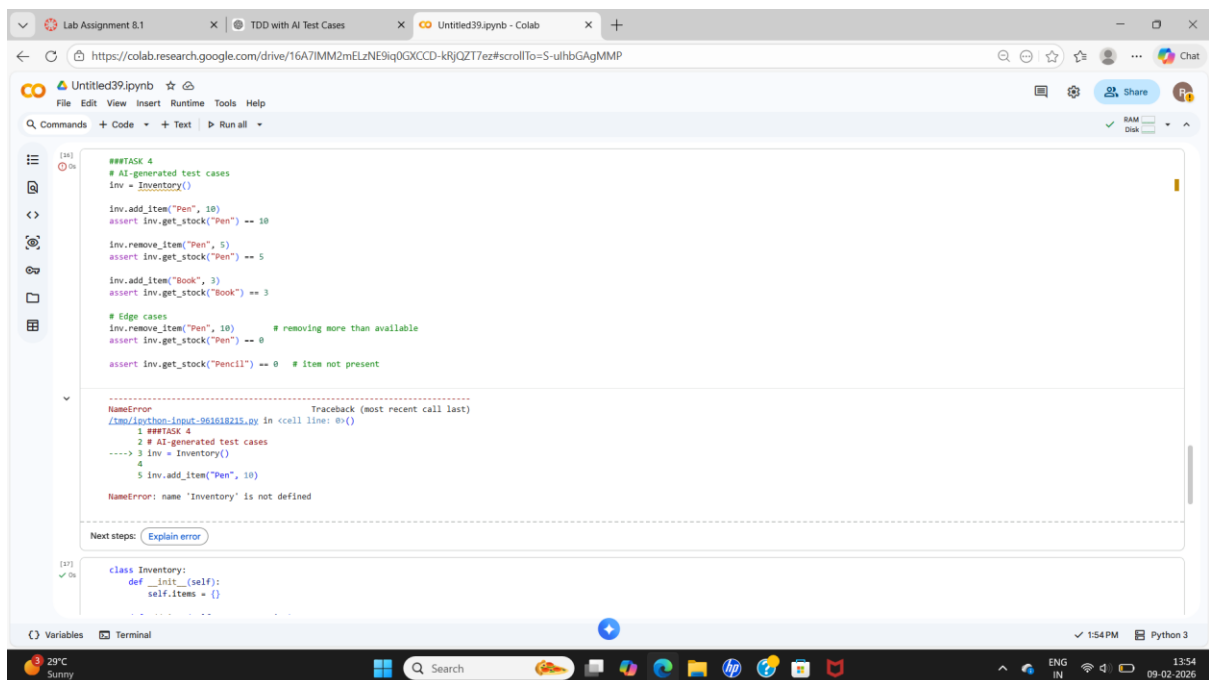
Code Explanation:

The function removes spaces and punctuation, converts strings to lowercase, and compares characters to check if two strings are anagrams.

AI Explanation:

AI-generated tests helped cover case differences, spaces, and empty strings for accurate string comparison.

Task 4: Inventory Class (Real-World Simulation)



The screenshot shows a Google Colab notebook with the following code in cell [146]:

```
##TASK 4
# AI-generated test cases
Inv = Inventory()

Inv.add_item("Pen", 10)
assert Inv.get_stock("Pen") == 10

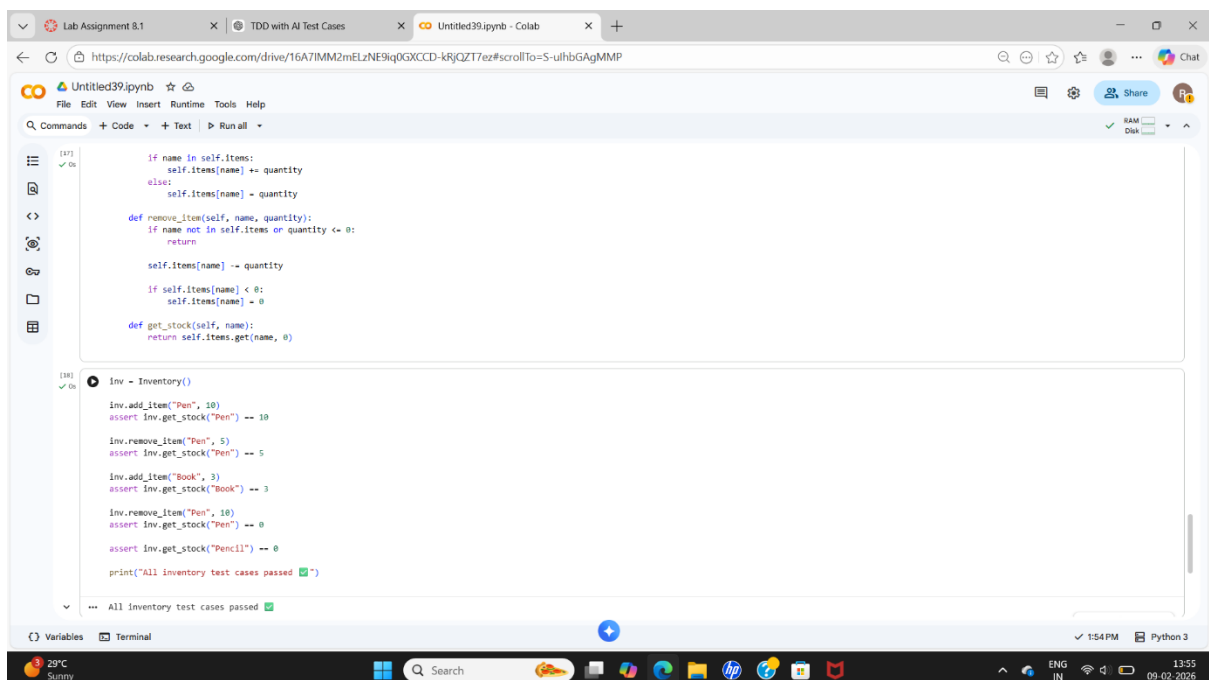
Inv.remove_item("Pen", 5)
assert Inv.get_stock("Pen") == 5

Inv.add_item("Book", 3)
assert Inv.get_stock("Book") == 3

# Edge cases
Inv.remove_item("Pen", 10) # removing more than available
assert Inv.get_stock("Pen") == 0

assert Inv.get_stock("Pencil") == 0 # item not present
```

The execution of this cell results in a `NameError` because the `Inventory` class has not been defined yet. The error message is: `NameError: name 'Inventory' is not defined`. Below the error, the "Next steps" section suggests clicking "Explain error".



The screenshot shows the same Google Colab notebook with the `Inventory` class implemented in cell [147]:

```
class Inventory:
    def __init__(self):
        self.items = {}

    def add_item(self, name, quantity):
        if name in self.items:
            self.items[name] += quantity
        else:
            self.items[name] = quantity

    def remove_item(self, name, quantity):
        if name not in self.items or quantity <= 0:
            return

        self.items[name] -= quantity

        if self.items[name] < 0:
            self.items[name] = 0

    def get_stock(self, name):
        return self.items.get(name, 0)
```

In cell [148], the test cases from the previous cell are repeated:

```
Inv = Inventory()

Inv.add_item("Pen", 10)
assert Inv.get_stock("Pen") == 10

Inv.remove_item("Pen", 5)
assert Inv.get_stock("Pen") == 5

Inv.add_item("Book", 3)
assert Inv.get_stock("Book") == 3

Inv.remove_item("Pen", 10)
assert Inv.get_stock("Pen") == 0

assert Inv.get_stock("Pencil") == 0

print("All inventory test cases passed")
```

The execution of cell [148] is successful, and the output is: `All inventory test cases passed`.

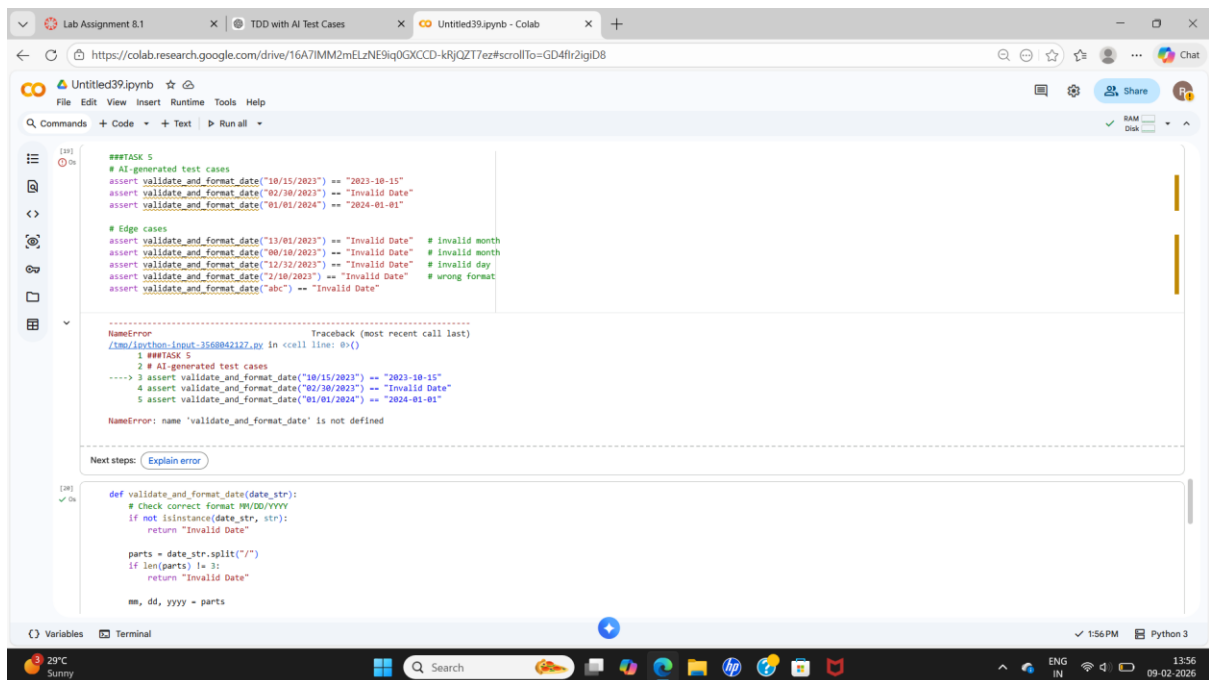
Code Explanation:

The Inventory class manages item stock. It allows adding items, removing items safely, and checking stock quantity.

AI Explanation:

AI provided test cases simulating real inventory actions like adding, removing, and checking items.

Task 5: Date Validation & Formatting



The screenshot shows a Jupyter Notebook interface with a tab titled 'Untitled39.ipynb - Colab'. The code cell contains a function definition for `validate_and_format_date` and a series of test cases. A `NameError` is displayed, indicating that the function name is not defined. The error message is: `NameError: name 'validate_and_format_date' is not defined`. The code cell shows the function definition and test cases, but the function is not yet fully implemented.

```
##TASK 5
# AI-generated test cases
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"

# Edge cases
assert validate_and_format_date("13/01/2023") == "Invalid Date" # invalid month
assert validate_and_format_date("00/10/2023") == "Invalid Date" # invalid month
assert validate_and_format_date("12/32/2023") == "Invalid Date" # invalid day
assert validate_and_format_date("2/10/2023") == "Invalid Date" # wrong format
assert validate_and_format_date("abc") == "Invalid Date"

NameError                                Traceback (most recent call last)
/mnt/tpch-jupyter-3560842327.py in <cell line: 0>()
      1 ##TASK 5
      2 # AI-generated test cases
----> 3 assert validate_and_format_date("10/15/2023") == "2023-10-15"
      4 assert validate_and_format_date("02/30/2023") == "Invalid Date"
      5 assert validate_and_format_date("01/01/2024") == "2024-01-01"

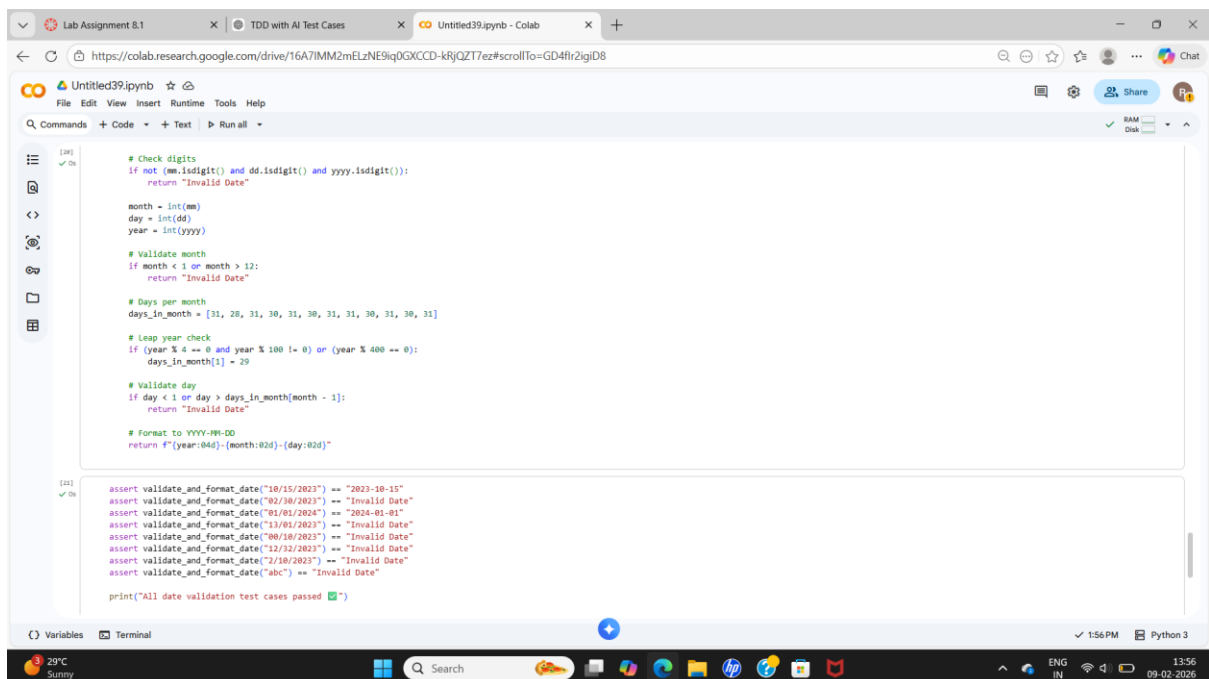
NameError: name 'validate_and_format_date' is not defined

Next steps: Explain error
```

```
def validate_and_format_date(date_str):
    # Check correct format MM/DD/YYYY
    if not isinstance(date_str, str):
        return "Invalid Date"

    parts = date_str.split("/")
    if len(parts) != 3:
        return "Invalid Date"

    mm, dd, yyyy = parts
```



The screenshot shows the same Jupyter Notebook interface, but the function `validate_and_format_date` is now fully implemented. The code cell contains the function definition and test cases. The function checks the format, validates the date, and converts valid dates to YYYY-MM-DD format. The test cases are executed, and the output shows that all test cases passed.

```
# Check digits
if not (mm.isdigit() and dd.isdigit() and yyyy.isdigit()):
    return "Invalid Date"

month = int(mm)
day = int(dd)
year = int(yyyy)

# Validate month
if month < 1 or month > 12:
    return "Invalid Date"

# Days per month
days_in_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

# Leap year check
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    days_in_month[1] = 29

# Validate day
if day < 1 or day > days_in_month[month - 1]:
    return "Invalid Date"

# Format to YYYY-MM-DD
return f"{year:04d}-{month:02d}-{day:02d}"

assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"
assert validate_and_format_date("13/01/2023") == "Invalid Date"
assert validate_and_format_date("00/10/2023") == "Invalid Date"
assert validate_and_format_date("12/32/2023") == "Invalid Date"
assert validate_and_format_date("2/10/2023") == "Invalid Date"
assert validate_and_format_date("abc") == "Invalid Date"

print("All date validation test cases passed")
```

Code Explanation:

The function checks whether a date is in MM/DD/YYYY format, validates the date, and converts valid dates to YYYY-MM-DD.

AI Explanation:

AI generated test cases for valid dates, invalid formats, and edge cases like leap years.

